

Research on the application of artificial intelligence in software automation test case generation in aerospace field

Wenquan Li*

XiChang Latellite Launch Center
XiChang, Sichuan, China
820021257@qq.com

Fangsong Shao

XiChang Latellite Launch Center
XiChang, Sichuan, China
1620651774@qq.com

Yu Fang

XiChang Latellite Launch Center
XiChang, Sichuan, China
fangyu_oec@163.com

Abstract

With the rapid development of the aerospace industry, the reliability and safety of aerospace software are becoming more and more critical. Traditional software testing methods have been difficult to meet their complex testing requirements. In recent years, the rise of artificial intelligence technology has brought new development opportunities to the field of software testing. The purpose of this study is to analyze the current situation of software automation test case generation and application of artificial intelligence in the aerospace field, and to explore its potential advantages and challenges. In order to provide theoretical support and reference for the wide application of artificial intelligence in the field of aerospace software automation testing in the future, it will help to improve the efficiency and coverage of test case generation, improve the quality of aerospace software, and ensure the smooth implementation of aerospace missions.

CCS Concepts

• **Software and its engineering** → Software organization and properties; Software functional properties..

Keywords

aerospace software, reliability, automated testing, artificial intelligence, test cases

ACM Reference Format:

Wenquan Li, Fangsong Shao, and Yu Fang. 2025. Research on the application of artificial intelligence in software automation test case generation in aerospace field. In *2025 9th International Conference on Electronic Information Technology and Computer Engineering (EITCE 2025)*, June 13–15, 2025, Chongqing, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3766671.3766826>

1 Introduction

1.1 Background and Significance of research

As a critical component of national strategic development, the aerospace industry has a profound impact on the country's scientific and technological strength, national defense security and international competitiveness. From the first time that human beings step into space, to today's various complex space missions,

such as deep space exploration, manned space flight, satellite networking, etc., every progress in space technology is inseparable from the support of software systems. Aerospace software is responsible for controlling the key tasks of spacecraft, such as flight attitude, orbit adjustment, data acquisition and transmission. Its reliability and security directly determine the success or failure of aerospace missions. Any minor software failure may lead to serious consequences. For example, the failure of the European Ariane 5 rocket launch in 1996 was due to an overflow error when the software converted the rocket flight data from a 64-bit floating-point number to a 16-bit integer, resulting in the rocket deviating from the orbit and self-destruction only 37 seconds after launch, resulting in a loss of up to USD 500 million[1]. This incident fully highlights the importance of aerospace software quality assurance.

Traditional aerospace software testing mainly relies on manually writing test cases and performing test case testing. Although manual testing can find some problems with the experience of testers, it has become inadequate in the face of the increasing scale and complexity of software in the aerospace field. On the one hand, manual writing test cases is inefficient and test case coverage is low; on the other hand, manually writing test cases is prone to omissions, which cannot meet the strict requirements of aerospace data processing software for accuracy and reliability, and is highly subjective and susceptible to human factors. With the rapid development of artificial intelligence technology in recent years, deep learning[2], natural language processing[3] and other technologies have achieved remarkable results in image recognition, speech recognition, intelligent recommendation and other fields, providing new ideas and methods for aerospace software automation testing. The introduction of artificial intelligence technology into aerospace software automated testing is expected to achieve intelligent generation of test cases, accurate prediction of defects, and efficient analysis of test results, thereby improving test efficiency, reducing test costs, and enhancing software reliability and security. It is of great significance to ensure the smooth implementation of aerospace missions.

1.2 Research status at home and abroad

The United States, Europe and other space powers and regions have been committed to the research and innovation of aerospace software testing technology. The National Aeronautics and Space Administration (NASA) began to explore the application of artificial intelligence technology in aerospace software testing many years ago[4]. They use machine learning algorithms to analyze the historical test data of aerospace software, establish a defect prediction model, and discover potential software problems in advance. The European Space Agency (ESA) has done a lot of research

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *EITCE 2025, Chongqing, China*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1404-7/2025/06
<https://doi.org/10.1145/3766671.3766826>

on test case generation, and generated efficient test cases through intelligent search algorithms such as genetic algorithms to improve test coverage[5]. In addition, some internationally renowned research institutions and enterprises are also actively carrying out relevant research, such as the Software Engineering Institute of Carnegie Mellon University, which has achieved many theoretical and practical results in the application of artificial intelligence in software testing.

In recent years, China has made great progress in applying artificial intelligence technology to aerospace software testing, such as using deep learning technology to realize automatic testing of image data software, and analyzing test requirements documents through natural language processing technology[6]. However, compared with the advanced level of foreign countries, there is still a big gap between domestic and foreign technology in aerospace software automation testing technology. Although domestic artificial intelligence technology and application have developed rapidly in recent years, there is still a long way to go to apply artificial intelligence technology to the aerospace industry, especially in the innovation of key technologies and the depth and breadth of practical application. There is still much room for improvement.

1.3 Subject and method

This research mainly focuses on the generation and application of test cases in artificial intelligence in software automation testing in the aerospace field. In terms of key technologies, in-depth study of intelligent test case generation technology to improve the efficiency and coverage of test case generation.

In terms of application, by selecting representative aerospace software functions for case analysis, this paper expounds the specific application of artificial intelligence technology in case generation in the process of software automation test, and comprehensively evaluates the application effect, and compares the changes of key indicators such as test case generation efficiency, test case coverage and test cost before and after the application of artificial intelligence.

This research mainly uses the literature research method to extensively collect literature on aerospace software testing, artificial intelligence technology applications, etc., to understand the research status and development trends in this field, and to provide a theoretical basis for research.

2 The problems faced by traditional software testing technology

2.1 Aerospace software features

Software in the aerospace field is no longer an accessory to hardware, but becomes the nerve center of the aerospace system [7]. Many key software shoulders the key missions of controlling spacecraft flight, carrier rocket control, data processing and transmission, and has distinct characteristics. High reliability is its most prominent feature. Due to the high cost of launching spacecraft and carriers, and the difficulty of manual maintenance once entering space, any minor fault of the software can cause catastrophic consequences. Therefore, aerospace software must have extremely high reliability to ensure long-term stable operation in complex and harsh space environments. For example, the satellite's attitude

control system software needs to accurately control the direction of the satellite to ensure that its solar panels are always aligned to the sun and the communication antenna is aligned to the earth. Once the software fails, the satellite will not work properly, resulting in mission failure.

Strong real-time performance is also an important feature of aerospace software. In the process of space mission execution, many operation delay requirements are in milliseconds, and the software needs to complete data processing and instruction response within the specified time. Taking manned spaceflight as an example, in the process of returning to the earth, the software must monitor the speed, height, attitude and other parameters of the spacecraft in real time, and send instructions to adjust the flight state in time to ensure the safe landing of the spacecraft. If the software response delay is too large, it may lead to the failure of the spacecraft landing and endanger the safety of astronauts.

Furthermore, aerospace software is highly complex. It involves the collaborative work of multiple systems, including orbit calculation, attitude control, information collection, data transmission, etc. The data interaction and control logic between different systems are complex.

2.2 The Importance of Software Testing in Aerospace Field

Software testing plays an important role in the aerospace field and is a key link to ensure the success of aerospace missions. From the past aerospace accidents, we can deeply understand the importance of software testing. In addition to the aforementioned launch failure of the European Ariane 5 rocket in 1996, there are still many cases of launch failure abroad. In 1998, the US Mars climate orbiter disappeared during the Mars exploration mission. The reason is that the ground control software uses the British unit, and the software on the spacecraft uses the metric unit. Due to the unit conversion error, the orbit calculation deviation is caused. Finally, the spacecraft crashes due to too low when entering the Mars orbit. This accident caused a loss of up to USD 193 million[8]. These painful lessons show that even seemingly insignificant software defects may cause serious consequences in aerospace missions, which not only lead to huge economic losses, but also may make years of scientific research efforts go to waste, and even endanger life safety.

The significance of software testing is to find potential defects and loopholes in the software as much as possible before the aerospace software is put into use through comprehensive and strict software testing, and to repair and optimize the defects in advance, so as to reduce the risk in the process of aerospace mission execution. Software testing can also verify whether the software meets the requirements, design requirements and functional specifications to ensure that the software can operate stably and reliably in a complex environment. In the field of aerospace, any point of uncertainty can bring catastrophic consequences. Software testing provides important quality assurance for the smooth implementation of space missions and is a necessary means to ensure the success of space missions.

2.3 Traditional software testing methods and challenges

Traditional software testing methods mainly include black box testing, white box testing and gray box testing. Black box testing is a kind of software product as a black box, without understanding the internal structure and implementation details of the software, by inputting different test data, observe the output results of the software to determine whether the software meets the expected function. For example, for a spacecraft communication software, black box testing will simulate different communication scenarios, such as signal strength changes, data transmission rate changes, etc., to test whether the software can correctly send and receive data. White-box testing is based on the source code and internal program structure of the software, and tests the internal logic, path, branch, and statement coverage of the program to ensure that the internal implementation logic of the software is correct. For example, by analyzing the source code of flight control software, different test cases are designed to cover different control logic branches and check the execution of the code. Grey box testing combines the characteristics of black box testing and white box testing. It not only pays attention to the functional performance of the software, but also understands some internal structures and conducts targeted testing.

However, these traditional testing methods face many challenges in the face of increasingly complex aerospace software systems. In terms of test efficiency, with the increasing scale and complexity of aerospace software, traditional test methods require a lot of time and manpower to design and execute test cases, which is difficult to meet the needs of fast iterative software development. In terms of test coverage, it is almost impossible to ensure that all possible input combinations and operating scenarios are fully tested. Traditional test methods are difficult to achieve high coverage and easily miss potential defects. For the processing ability of complex scenes, the operating environment of aerospace software is complex and changeable, involving a variety of physical phenomena and system interactions. Traditional testing methods are difficult to simulate these complex scenes, and cannot effectively detect the performance and stability of software in complex environments. In addition, the analysis of test results by traditional test methods mainly depends on manual judgment, which is subjective and difficult to extract valuable information from massive test data quickly and accurately, and to find problems in software in time.

3 The development of artificial intelligence in the field of software testing

In recent years, under the wave of rapid development of artificial intelligence in China, artificial intelligence has also shown a booming trend in the field of software testing, and the intelligent transformation in the field of software testing is accelerating.

Many domestic enterprises are actively engaged in the development and application of artificial intelligence software testing technology[9]. Baidu's AI automated testing platform uses artificial intelligence technology to provide intelligent automated testing services for mobile applications. The platform can intelligently update test scripts according to changes in applications, improve the maintainability and coverage of automated testing,

and greatly improve test efficiency. The test platform based on artificial intelligence technology launched by Huawei Cloud also provides services such as automated testing, performance testing and security testing. It uses machine learning and natural language processing technology[10] to help test teams manage and execute test cases more intelligently, effectively reducing test costs and improving software quality.

In terms of technology application, AI has played an important role in many key aspects of software testing. In terms of test case generation, based on deep learning and natural language processing technology, AI can analyze requirement documents, architecture design, code implementation, etc., and realize end-to-end generation and optimization from requirement documents to test cases. Some tools use large language models[11] to quickly generate a large number of test cases based on simple requirement descriptions, which greatly improves the speed of test case creation. It can also cover more edge scenarios, reduce the workload of manual scripting, and reduce the cost of test writing and maintenance.

4 Application of Artificial Intelligence in Software Automation Test Case Generation

4.1 Test requirement analysis and automatic test case generation

In traditional test methods, the design and optimization of test cases often consume a lot of time for testers, and the increasingly mature artificial intelligence technology provides us with a feasible solution to automatically generate and optimize test cases[12]. The deep learning function of artificial intelligence itself can analyze and detect problems in software through its own learning. Combined with natural language processing technology (NLP), it preprocesses multi-dimensional data such as software requirements specification documents or maintenance plans and design specifications, analyzes and sorts out input documents, extracts key information, identifies potential test scenarios, and generates targeted test cases. It enables traditional software testing and artificial intelligence to be well combined to help solve some single and complex problems manually, making software testing faster, simpler and more reliable with the help of artificial intelligence technology[13].

Taking the dual-machine hot backup function of the aerospace software system as an example, as the core module to ensure the continuous operation of the system, it is directly related to the success or failure of the aerospace mission. In the traditional test mode, the test of the dual-machine hot backup function requires manual construction of test scenarios such as fault injection, master-slave switching, and data synchronization. There are problems such as incomplete test case coverage and low execution efficiency. Table 1 shows the requirement analysis and test case generation process of dual-machine hot backup function based on artificial intelligence technology. By extracting key information such as 'dual-machine' and 'hot backup', the test case set covering normal switching, fault switching, performance verification and other dimensions is automatically constructed.

Table 1: Requirement analysis example.

requirement description	Key information extraction	test cases generation
Users need to test the switching function of Double machine, hot backup dual hot backup system.		Normal manual switching (main → standby) Host fault automatic switching (A fault → B takeover) Data consistency verification after switching Switching in network jitter scenarios Multiple switching Service recovery time after handover (RTO) Standby machine failure, the host is not affected

Table 2: Test data generation example.

Test Case Description	Prefix Condition	Generate test data
Test normal manual switching (main → standby)	The main and standby machine runs normally, and the perception between the main and standby is normal.	Click ' Switchover '
Test host fault automatic switching (A fault → B takeover)		Trigger host (A) failure
Data consistency verification after test switching		Perform manual or automatic switching (main → standby)
Test the switching in the network jitter scenario		Simulate network jitter
Test multiple switching		Execute the switch command every 30 seconds, 5 times in a row
Test the service recovery time (RTO) after switching.		Trigger host (A) failure
Test standby machine failure, the host is not affected		Trigger standby machine (B) failure

4.2 Generation of test data

In the traditional test data generation method, testers manually write data, and design input data and expected results for multiple situations for the same functional test. It is not only time-consuming and laborious, but also prone to human error, and it is difficult to fully cover complex and diverse test scenarios. Artificial intelligence technology has been widely used in the data generation of software testing, which has greatly changed this situation. The intelligent model can extract key information through key technologies according to the natural language description of the use case, and then generate the corresponding test data for each use case according to the key information.

In practical applications, the advantages of artificial intelligence in generating test data are very obvious. For example, in the parameter setting, the test data that can trigger the boundary condition and the test data that verify the input box 's legality check of the input data are automatically generated, thereby covering more test scenarios and improving the comprehensiveness of the test. Taking the dual-machine hot backup switching function as an example, artificial intelligence technology can generate targeted test data according to the preconditions and expected goals of different test cases. Table 2 shows the test data generation of typical test cases, and comprehensively verifies the rationality and reliability of system switching logic by simulating scenarios such as abnormal state of host and standby computers and network faults.

Artificial intelligence shows the characteristics of high efficiency, precision and flexibility in the process of test data generation. It not only solves the limitations of traditional methods, but also improves the diversity and effectiveness of test data by means of automation, which provides strong support for software quality assurance.

4.3 Test script generation

This study preprocesses the natural language description of test cases, extracts key information, and generates test scripts for corresponding test cases based on the extracted key information, which provides great help for generating test scripts in automated testing. Test script example of use case 1.

4.4 Test script generation

In order to verify the effectiveness of artificial intelligence in the preparation of software automation test, this study selected an aerospace internal software for comparative experiments. The experiment uses the traditional manual test method and the automatic test method based on artificial intelligence to compare and analyze the efficiency and coverage of the three core links of test case generation, test data generation and test script generation. The experimental results are shown in Table 3. The test method based on artificial intelligence shows significant advantages in all aspects : the test case generation time is shortened from 16 hours to 4.5 hours, and the efficiency is improved by 71.9 %. The test case coverage increased from 65 % to 89 %, an increase of 24 %. The test

Algorithm 1 def test_manual_switchover_normal(dual_system):

```

primary_status =
dual_system.get_system_status(dual_system.primary_url)
assert primary_status["status"] == "active", "primary_status is abnormal"
backup_status =
dual_system.get_system_status(dual_system.backup_url)
assert backup_status["status"] == "standby", "backup_status is abnormal"
# --- carry out switch ---
start_time = time.time()
switch_result = dual_system.manual_switchover()
switch_time = time.time() - start_time
# --- verification result ---
assert switch_result is True, "manual switch failed"
assert switch_time < 5, f"The switch time {switch_time:.2f} Sec, exceed 5Sec"
new_primary_status =
dual_system.get_system_status(dual_system.backup_url)
assert new_primary_status["status"] == "active", "Backup machine upgrade failed"
old_primary_status =
dual_system.get_system_status(dual_system.primary_url)
assert old_primary_status["status"] == "standby", "The downgrade of the ori_host failed"
business_response =
requests.get(f"{dual_system.current_primary}/business")
assert business_response.status_code == 200, "Business request failed"
assert "data" in business_response.json(), "Data exception"

```

data generation time is reduced from 10 hours to 2 hours, and the efficiency is increased by 75 %. The test script generation time is shortened from 10 hours to 3 hours, and the efficiency is improved by 70 %. These data show that artificial intelligence technology can effectively optimize the test preparation process and improve the overall efficiency of the test work.

4.5 Applicative Prospect

The application of intelligent test case generation technology in the field of software testing has emerged and has great application value[14]. In terms of test efficiency, in the traditional manual test mode, it takes several weeks to complete a comprehensive test. After the introduction of artificial intelligence technology, the test case generation and execution cycle are greatly shortened, which significantly accelerates the software iterative development process.

In terms of test cost, although certain R & D and computing resource costs need to be invested in the early stage of introducing artificial intelligence technology, in the long run, due to the improvement of test efficiency and the enhancement of defect discovery ability, the cost of software maintenance and fault repair in the later stage is reduced. At the same time, the increase of automated testing also reduces the dependence on a large number of testers, and the overall test cost is also greatly reduced.

Especially in the aerospace field, the high reliability and safety requirements of software systems are extremely stringent. Artificial intelligence-driven automated testing technology can effectively cope with the testing challenges in complex scenarios and ensure the smooth implementation of aerospace missions. Therefore, this technology has broad application prospects and promotion value in aerospace software testing, and is expected to become the core technical means of future aerospace software development quality assurance.

5 Application Challenges and Solutions

5.1 Data quality and security issues

Aerospace software testing data sources are extensive, including software code libraries, test execution records, spacecraft sensor data, etc. In the process of collecting and collating these data, incomplete data often occurs, such as the time stamp of some test execution records missing key steps, or the loss of some data points due to signal interference during the transmission of sensor data. Inaccurate data is also a common problem. For example, when the code measurement tool calculates the code complexity, the result is wrong due to the analytical deviation of the complex algorithm. Low-quality data will seriously affect the training effect of artificial intelligence model, so that the model can learn the wrong patterns and characteristics, which may lead to the problem of unreasonable and inaccurate test case generation. Of course, the test data generated by the intelligent model system may also have a small amount of quality problems, resulting in incorrect input during the execution of automated testing, which requires manual optimization and correction to ensure accurate and complete data.

Software and test data in the aerospace field usually contain sensitive information such as core algorithms, orbital parameters, and control instructions. There are strict confidentiality requirements, and the use of artificial intelligence may pose serious security threats. Data is at risk of being stolen and tampered during storage and transmission.

In order to solve the problem of data quality, it is necessary to establish a strict data preprocessing process. In the data collection stage, the data verification mechanism is set up to check the integrity and accuracy of the data in real time, and mark and repair

Table 3: Comparison between traditional methods and artificial intelligence.

index	traditional method	Artificial intelligence	efficiency change
Test case generation time	16h	4.5h	Time is shortened by 71.9 %.
Test case coverage rate	65%	89%	Coverage increased by 24 %
Test data generation time	10h	2.5h	Time is shortened by 75 %
Test script generation time	10h	3h	Time is shortened by 70 %

the missing or wrong data in time. The data cleaning algorithm is used to remove duplicate and noise data, and the abnormal values are reasonably processed. In terms of data security, artificial intelligence tools adopt the field privatization deployment method, and the model adopts local training and is isolated from physical means.

5.2 Model adaptability and interpretability

There are many types of aerospace software, and the functions, architecture and operating environment of different software are very different. For example, satellite navigation software is quite different from multi-source data processing software in algorithm complexity, real-time requirements and data processing methods. Current artificial intelligence models are often trained for specific types of software or data sets. When applied to different types of aerospace software testing, the adaptability of the model is poor, and it is difficult to accurately generate test cases.

Artificial intelligence models such as deep learning models are usually regarded as 'black boxes', and their internal decision-making processes and mechanisms are difficult to understand. In aerospace software testing, this is a serious problem. For example, when the model predicts that a software module is defective, testers cannot determine which features and factors the model is based on, and it is difficult to determine the root cause of the defect and the solution. This makes testers less trust in the model and limits the wide application of artificial intelligence technology in aerospace software testing.

In order to improve the adaptability of the model, transfer learning technology can be used. Firstly, pre-training is performed on the large-scale general aerospace software dataset to learn common software features and patterns. Then, for specific aerospace software projects, a small amount of project-specific data is used for fine-tuning, so that the model can quickly adapt to new software testing requirements. For the problem of model interpretability, on the one hand, it studies interpretable artificial intelligence algorithms, such as decision trees, rule-based models, etc. The decision-making process of these models is relatively intuitive and easy to understand. On the other hand, a visualization tool is developed to visualize the internal feature extraction and decision-making process of the deep learning model to help testers understand the behavior and decision-making basis of the model.

5.3 The problem of talent and technology integration

The aerospace field has extremely high professional requirements for software testers. It not only needs to master the theory and method of software testing, but also needs to understand the working principle, orbital mechanics, space environment and other knowledge of the aerospace system. In order to completely replace human beings with artificial intelligence for software testing, it is necessary for artificial intelligence to have the basis of strong artificial intelligence. The introduction of artificial intelligence technology requires testers to have knowledge and skills in machine learning and deep learning. At present, there is a serious shortage of compound talents who know both aerospace software testing and artificial intelligence technology. This leads to obstacles in

technical understanding and application when applying artificial intelligence technology to aerospace software automation testing, and cannot give full play to the advantages of artificial intelligence technology. For example, when implementing a defect prediction model based on machine learning, the testers do not have a deep understanding of the machine learning algorithm, and cannot reasonably select and adjust the model parameters, which affects the performance of the model.

In order to solve the problem of talent shortage, universities and research institutions should strengthen the setting of relevant professional courses and cultivate compound talents with aerospace knowledge and artificial intelligence technology. Enterprises and aerospace institutions can organize internal training and technical exchange activities to improve the level of artificial intelligence technology of existing testers.

6 Conclusions

This research focuses on the exploration of the generation and application of test cases in artificial intelligence in aerospace software automation testing. Artificial intelligence technology is initially applied to the design and generation process of software test cases in the aerospace field, which effectively improves the generation efficiency and coverage of test cases. With the help of artificial intelligence technology, more comprehensive and more targeted test cases can be generated for the complex functions and boundary conditions of aerospace software, which greatly reduces the labor cost and improves the efficiency of software testing as a whole.

In summary, the application of artificial intelligence technology in the field of software testing has great potential, which can greatly improve the intelligence level of software testing process in the aerospace field and solve the shortcomings and challenges faced by traditional software testing. At the same time, to promote the intelligent development of software testing field, it will also face many technical challenges to be solved, such as data acquisition and processing, data security and security, model training and optimization. In the future, with the continuous development and maturity of domestic artificial intelligence technology, it is believed that more innovative methods and applications will also come.

References

- [1] Leveson, N. G. (2004). The Role of Software in Spacecraft Accidents. *Journal of Spacecraft and Rockets*, 41(4), 564-575.
- [2] Xinyu Su, research on automatic testing method of interlocking software based on deep learning [D], Beijing Jiaotong University, 2023.DOI : 10.26944 / d.cnki.gbju.2023.003205.
- [3] Hong Zou. Research on the application of natural language processing technology based on artificial intelligence in software testing [J]. *Network security and informatization*, 2024, (04) : 59-61.
- [4] Williams, R., & Zhang, L. (2023). "Explainable AI for NASA's Spacecraft Software Testing: A Case Study on Artemis Program" *Vol.66(5)*, 205301
- [5] c. Almulla, H., & Gay, G. (2023). "Learning-Guided Search-Based Testing for Complex Space Systems." *Empirical Software Engineering*, 28(2), 45.
- [6] Gestí Nader. Research on software testing automation strategy based on artificial intelligence [J]. *Information and Computer*, 2025,37 (06) : 66-68.
- [7] Xuehua Zhou. Research on aerospace software testing technology [J]. *Journal of North China Institute of Aerospace Technology*, 2020,30 (06) : 17-19.
- [8] Johnson, M., & Li, W. (2021). "Lessons from Mars Climate Orbiter: A Systematic Review of Unit Conversion Errors in Aerospace Systems" *Vol. 34(3)*, 456-470.
- [9] Qifang Yang, Baorong Zhan. Development and Application of Computer Software Based on Artificial Intelligence [J]. *High Technology and Industrialization*, 2024, 30 (11) : 64-66

- [10] Yaping Yang. Application of artificial intelligence-based natural language processing technology in software testing [J]. Information recording materials, 2023,24 (11) : 97-99.
- [11] Jingxin Sun. Research on the application of artificial intelligence in the software automation testing industry [J].Zhang Jiang Technology Review, 2024, (03) : 64-66.
- [12] Chuan Zhang, Hongchang Xu, Tianjiao Luo, Research on the application of artificial intelligence technology in the field of software testing [N], Shanxi Science and Technology Journal, 2024-07-09 (A03).DOI : 10.28712 / n.cnki.nshxk.2024.000254.
- [13] Dongming Zhao, Linxiao Zhang, Wenhua Zhang. Research on the application of software testing technology under the background of artificial intelligence [J].Information and computer (theoretical version), 2020,32 (23) : 132-133.
- [14] Dong Wang. Application of Artificial Intelligence Technology in Software Testing [J]. Modern Computers, 2023, 29 (12) : 55-59